Project 01

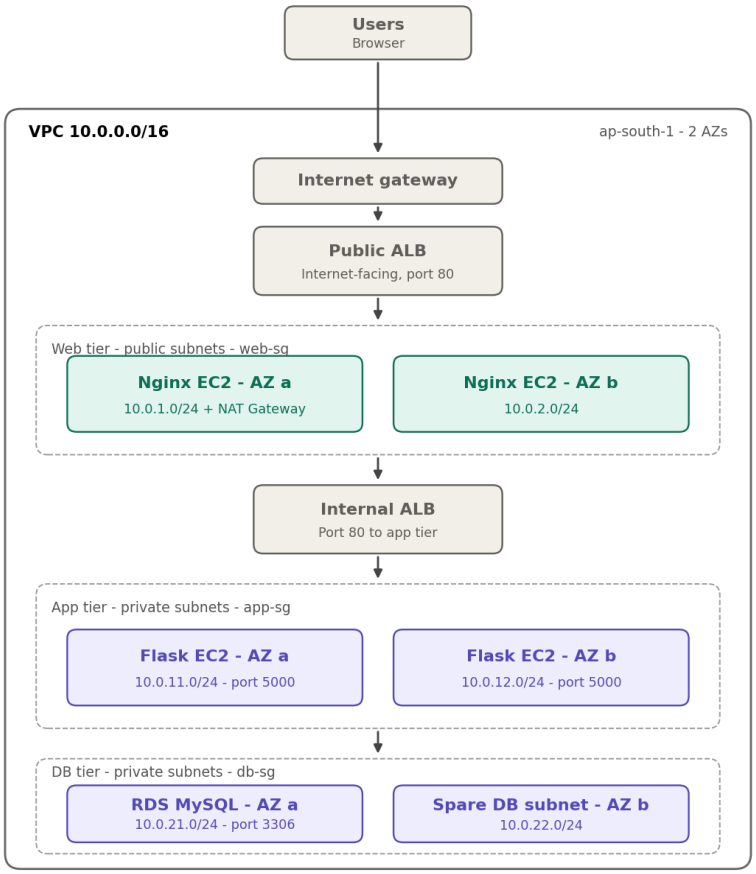
Building Scalable, Secure, and High-Performance Web Applications with AWS 3-Tier Architecture

Implementation report

Prepared by: Pandi Selvam P
Region: Asia Pacific (Mumbai) - ap-south-1
Date: 24 /10/ 2026

1. Overview and architecture

This project deploys a feedback web application on AWS using a three-tier design. Each tier sits in its own subnets and accepts traffic only from the tier above it. The setup spans two Availability Zones (ap-south-1a and ap-south-1b).



Request flow

User browser, then Internet Gateway, Public ALB, Nginx web servers, Internal ALB, Flask application servers, and finally the RDS MySQL database.

Tier	Components	Subnets
Web	Nginx on EC2 (Ubuntu 24.04, t3.micro), public ALB	Public: 10.0.1.0/24, 10.0.2.0/24
Application	Flask on EC2, internal ALB	Private: 10.0.11.0/24, 10.0.12.0/24
Database	RDS MySQL 8.4 (db.t3.micro, single-AZ)	Private: 10.0.21.0/24, 10.0.22.0/24

2. Task 1 - VPC and networking

Create an isolated network with public and private subnets across two Availability Zones.

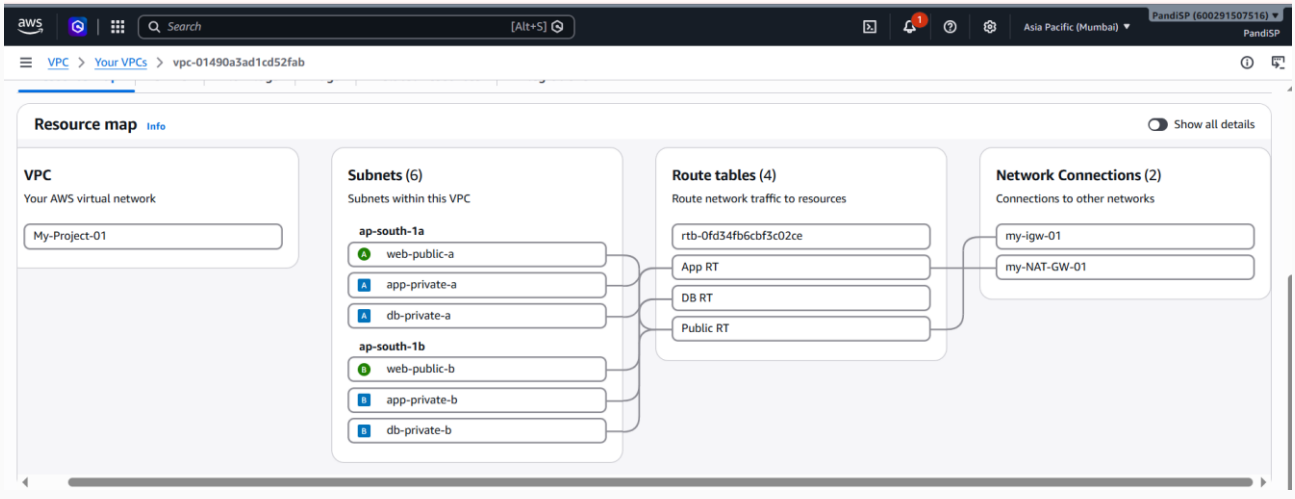
Steps performed

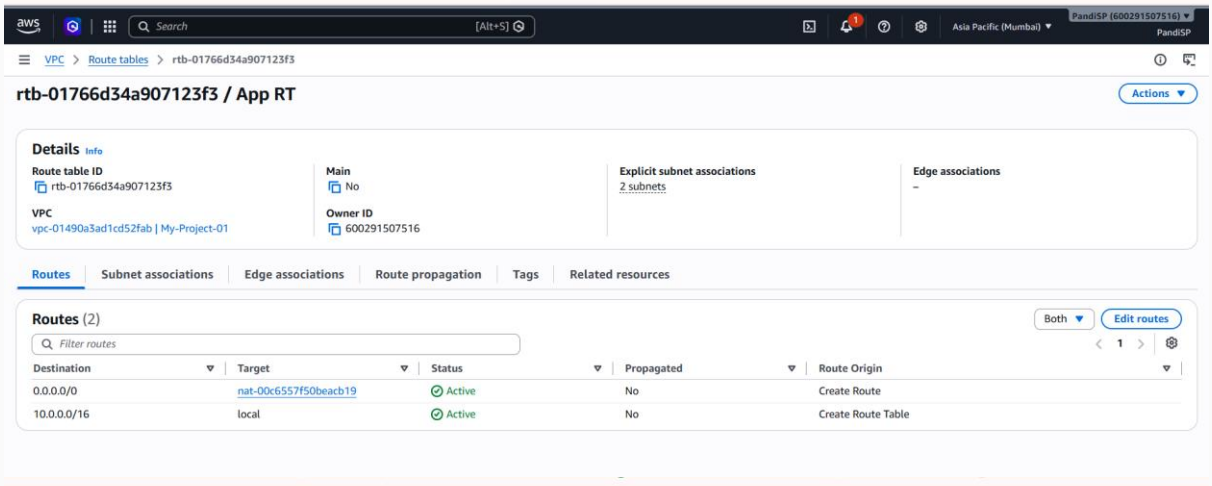
- Created VPC My-Project-01 with CIDR 10.0.0.0/16.
- Created six subnets (two public web, two private app, two private database) across ap-south-1a and ap-south-1b; enabled auto-assign public IPv4 on the web subnets.
- Created and attached the Internet Gateway (my-igw-01).
- Allocated an Elastic IP and created a zonal NAT Gateway (my-NAT-GW-01) in web-public-a.
- Created three route tables: Public RT (0.0.0.0/0 to the Internet Gateway), App RT (0.0.0.0/0 to the NAT Gateway) and DB RT (local route only), and associated the subnets with them.

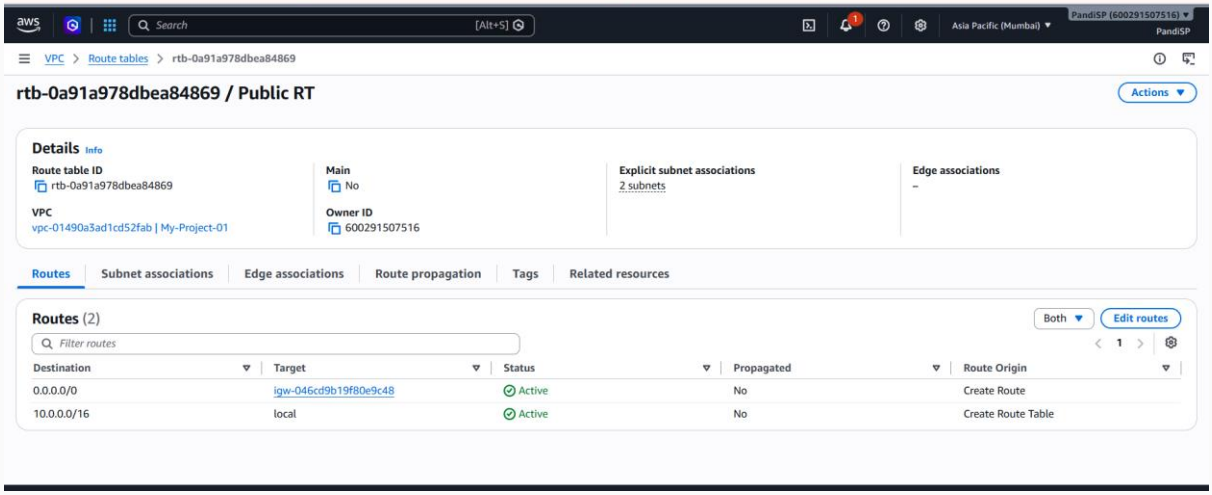
Subnet plan

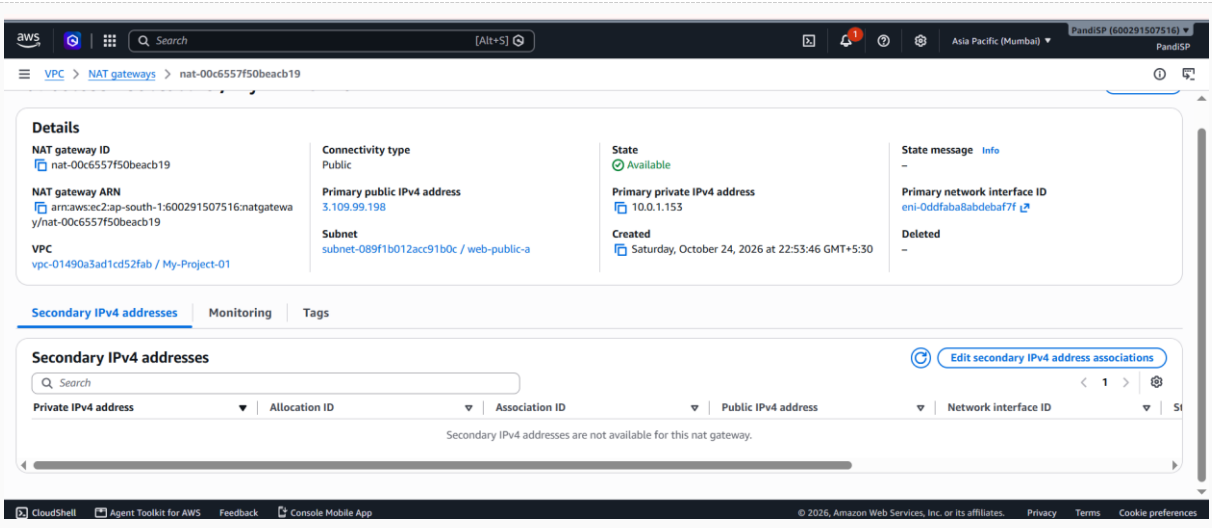
Subnet	Availability Zone	CIDR	Route table
web-public-a	ap-south-1a	10.0.1.0/24	Public RT
web-public-b	ap-south-1b	10.0.2.0/24	Public RT
app-private-a	ap-south-1a	10.0.11.0/24	App RT
app-private-b	ap-south-1b	10.0.12.0/24	App RT
db-private-a	ap-south-1a	10.0.21.0/24	DB RT
db-private-b	ap-south-1b	10.0.22.0/24	DB RT

Evidence




VPC > Route tables > rtb-01766d34a907123f3
rtb-01766d34a907123f3 / App RT
Actions
Details Info
Route table ID
rtb-01766d34a907123f3
Main
No
Explicit subnet associations
2 subnets
Edge associations
–
VPC
vpc-01490a3ad1cd52fab | My-Project-01
Owner ID
600291507516
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (2)
Both
Edit routes
Filter routes
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
nat-00c6557f50beacb19
Active
No
Create Route
10.0.0.0/16
local
Active
No
Create Route Table


VPC > Route tables > rtb-0a91a978dbea84869
rtb-0a91a978dbea84869 / Public RT
Actions
Details Info
Route table ID
rtb-0a91a978dbea84869
Main
No
Explicit subnet associations
2 subnets
Edge associations
–
VPC
vpc-01490a3ad1cd52fab | My-Project-01
Owner ID
600291507516
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (2)
Both
Edit routes
Filter routes
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
igw-046cd9b19f80e9c48
Active
No
Create Route
10.0.0.0/16
local
Active
No
Create Route Table


VPC > NAT gateways > nat-00c6557f50beacb19
Details
NAT gateway ID
nat-00c6557f50beacb19
Connectivity type
Public
State
Available
State message Info
–
NAT gateway ARN
arn:aws:ec2:ap-south-1:600291507516:natgateway/nat-00c6557f50beacb19
Primary public IPv4 address
3.109.99.198
Primary private IPv4 address
10.0.1.153
Primary network interface ID
eni-0ddfaba8abdebaf7f
VPC
vpc-01490a3ad1cd52fab / My-Project-01
Subnet
subnet-089f1b012acc91b0c / web-public-a
Created
Saturday, October 24, 2026 at 22:53:46 GMT+5:30
Deleted
–
Secondary IPv4 addresses
Monitoring
Tags
Secondary IPv4 addresses
Edit secondary IPv4 address associations
Search
Private IPv4 address
Allocation ID
Association ID
Public IPv4 address
Network interface ID
Secondary IPv4 addresses are not available for this nat gateway.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

3. Task 2 - Web tier (Nginx on EC2)

Launch the web server in a public subnet and install Nginx.

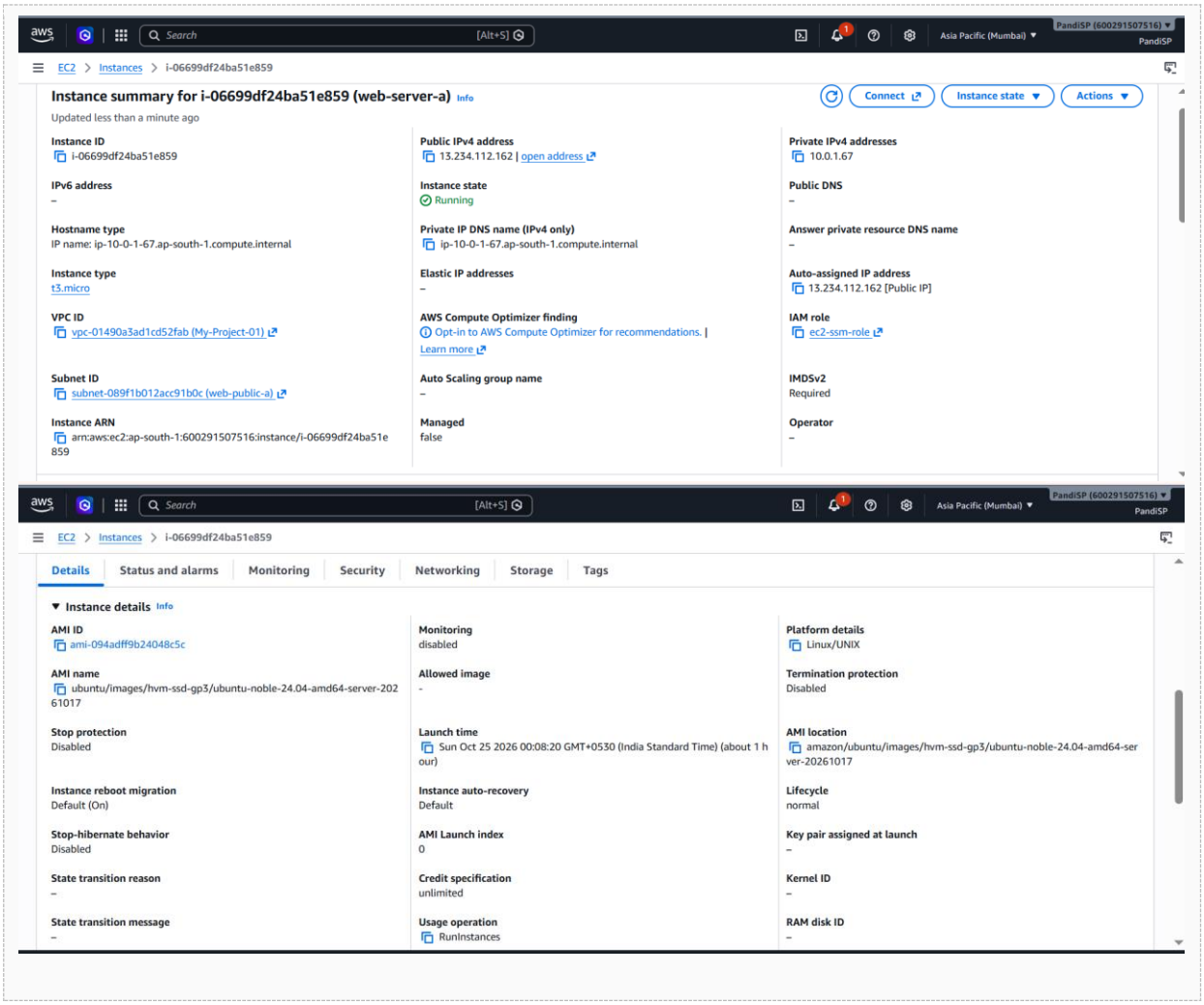
Steps performed

- Launched web-server-a: Ubuntu Server 24.04 LTS, t3.micro, subnet web-public-a, public IP enabled, security group web-sg, IAM role ec2-ssm-role.
- Connected with Session Manager and installed Nginx and Git.
- Cloned the project repository and copied index.html to /usr/share/nginx/html.

Key commands

```
sudo apt update
sudo apt install -y nginx git
git clone https://github.com/Vennilavanguvi/Three-Tier-Architecture.git
sudo mkdir -p /usr/share/nginx/html
sudo cp Three-Tier-Architecture/index.html /usr/share/nginx/html/
```

Evidence



```
ubuntu@ip-10-0-1-67:~$ sudo apt install -y nginx git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  git-man nginx-common
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
The following packages will be upgraded:
  git git-man
2 upgraded, 2 newly installed, 0 to remove and 15 not upgraded.
Need to get 5351 kB of archives.
After this operation, 1607 kB of additional disk space will be used.
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 git-man all 1:2.43.0-1ubuntu7.4 [1101 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 git amd64 1:2.43.0-1ubuntu7.4 [3681 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.18 [45.1 kB]
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx amd64 1.24.0-2ubuntu7.18 [525 kB]
Fetched 5351 kB in 0s (62.9 MB/s)
Preconfiguring packages ...
(Reading database ... 79249 files and directories currently installed.)
Preparing to unpack .../git-man_1%3a2.43.0-1ubuntu7.4_all.deb ...
Unpacking git-man (1:2.43.0-1ubuntu7.4) over (1:2.43.0-1ubuntu7.3) ...
```

a

```
Setting up nginx (1.24.0-2ubuntu7.18) ...
 * Upgrading binary nginx
Setting up git (1:2.43.0-1ubuntu7.4) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for ufw (0.36.2-6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-67:~$ git clone https://github.com/Vennilavanguvi/Three-Tier-Architecture.git
Cloning into 'Three-Tier-Architecture'...
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 21 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (21/21), 10.19 KiB | 10.19 MiB/s, done.
Resolving deltas: 100% (6/6), done.
ubuntu@ip-10-0-1-67:~$ sudo mkdir -p /usr/share/nginx/html
ubuntu@ip-10-0-1-67:~$ sudo cp Three-Tier-Architecture/index.html /usr/share/nginx/html/
ubuntu@ip-10-0-1-67:~$ sudo cp Three-Tier-Architecture/nginx.conf /etc/nginx/nginx.conf
ubuntu@ip-10-0-1-67:~$ grep -n "proxy_pass" /etc/nginx/nginx.conf
```

4. Task 3 - Application tier (Flask on EC2)

Launch the application server in a private subnet and run the Flask API.

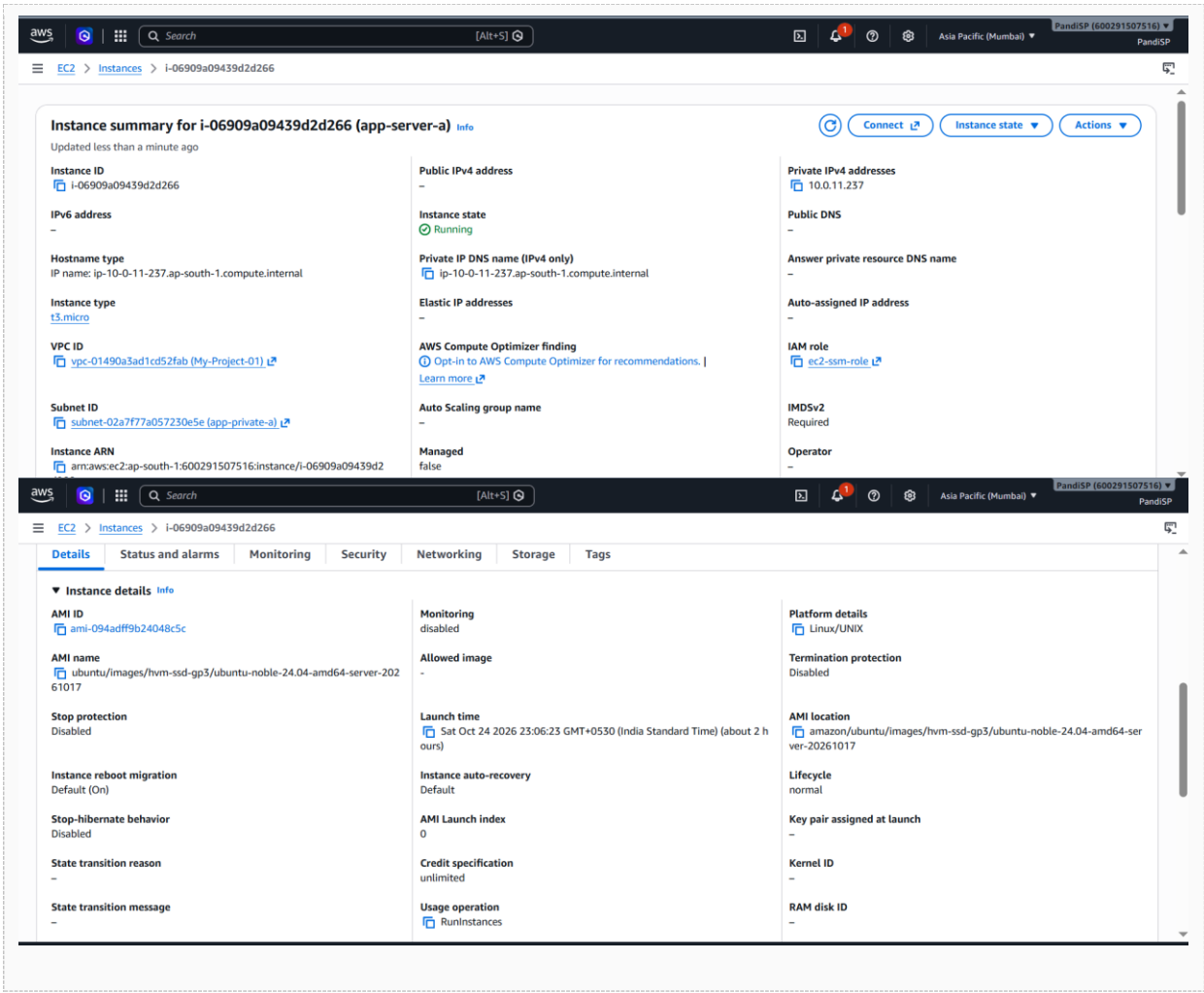
Steps performed

- Created IAM role ec2-ssm-role with AmazonSSMManagedInstanceCore so the private instance can be reached through Session Manager.
- Launched app-server-a in app-private-a with no public IP and security group app-sg.
- Installed Python packages in a virtual environment (Flask, PyMySQL, gunicorn).
- Copied app.py from the repository and verified the health endpoint on port 5000.

Key commands

```
sudo apt install -y python3-pip python3-venv python3-full git default-mysql-client
python3 -m venv venv && source venv/bin/activate
pip install Flask PyMySQL gunicorn
curl localhost:5000
# returns {"status":"Flask is running"}
```

Evidence



```
Oct 24 18:11:44 ip-10-0-11-237 systemd[1]: Started flask.service - Flask feedback app.
(venv) ubuntu@ip-10-0-11-237:~$ curl localhost:5000
curl -X POST localhost:5000/submit -H "Content-Type: application/json" -d '{"name":"Test","email":"t@t.com","feedback":"Hello RDS!"}'
{"status":"Flask is running"}
{"message":"Feedback submitted successfully!"}
```

5. Task 4 - Database tier (RDS MySQL)

Create a private MySQL database reachable only from the application tier.

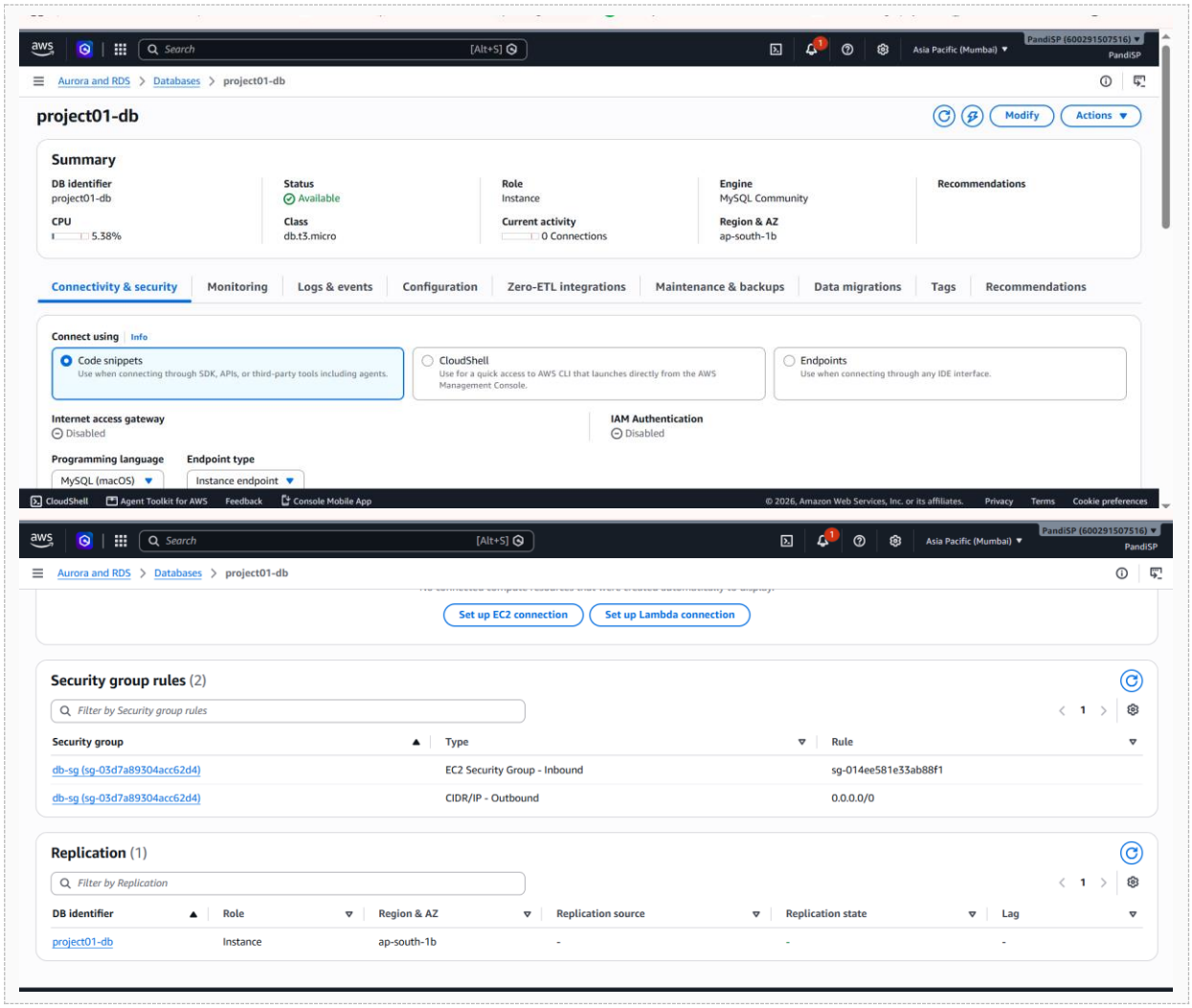
Steps performed

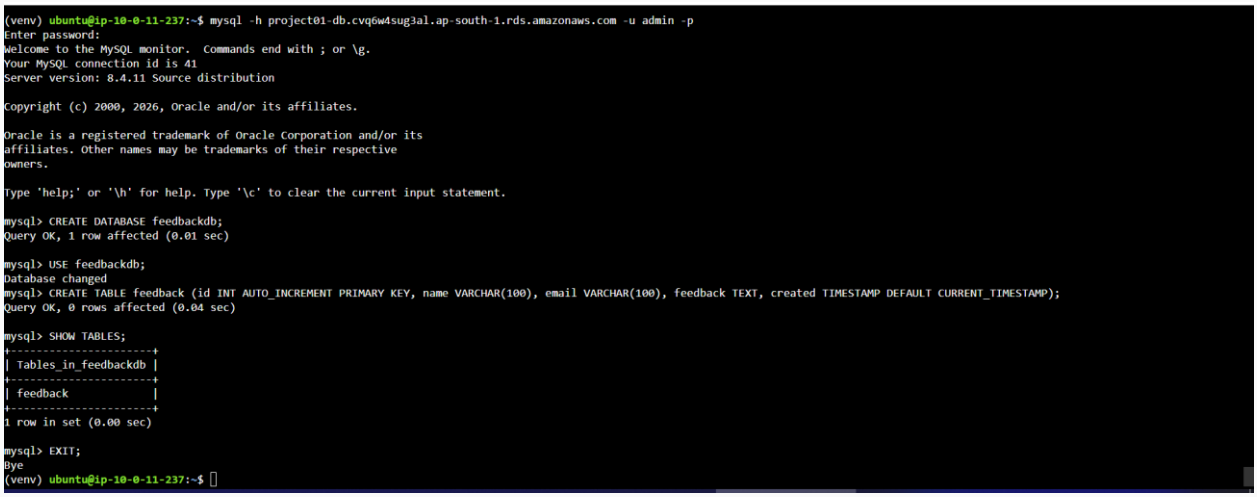
- Created DB subnet group db-subnet-group using db-private-a and db-private-b.
- Created RDS MySQL instance project01-db (db.t3.micro, single-AZ), public access disabled, security group db-sg.
- Connected from the app server with the MySQL client and created the feedbackdb database and feedback table.

Key commands

```
mysql -h <RDS-ENDPOINT> -u admin -p
CREATE DATABASE feedbackdb;
USE feedbackdb;
CREATE TABLE feedback (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), email
VARCHAR(100), feedback TEXT, created TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
SHOW TABLES;
```

Evidence




(venv) ubuntu@ip-10-0-11-237:~$ mysql -h project01-db.cvq6w4sug3al.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 41
Server version: 8.4.11 Source distribution
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE feedbackdb;
Query OK, 1 row affected (0.01 sec)
mysql> USE feedbackdb;
Database changed
mysql> CREATE TABLE feedback (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100), feedback TEXT, created TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.04 sec)
mysql> SHOW TABLES;
+----------------------+
| Tables_in_feedbackdb |
+----------------------+
| feedback |
+----------------------+
1 row in set (0.00 sec)
mysql> EXIT;
Bye
(venv) ubuntu@ip-10-0-11-237:~$

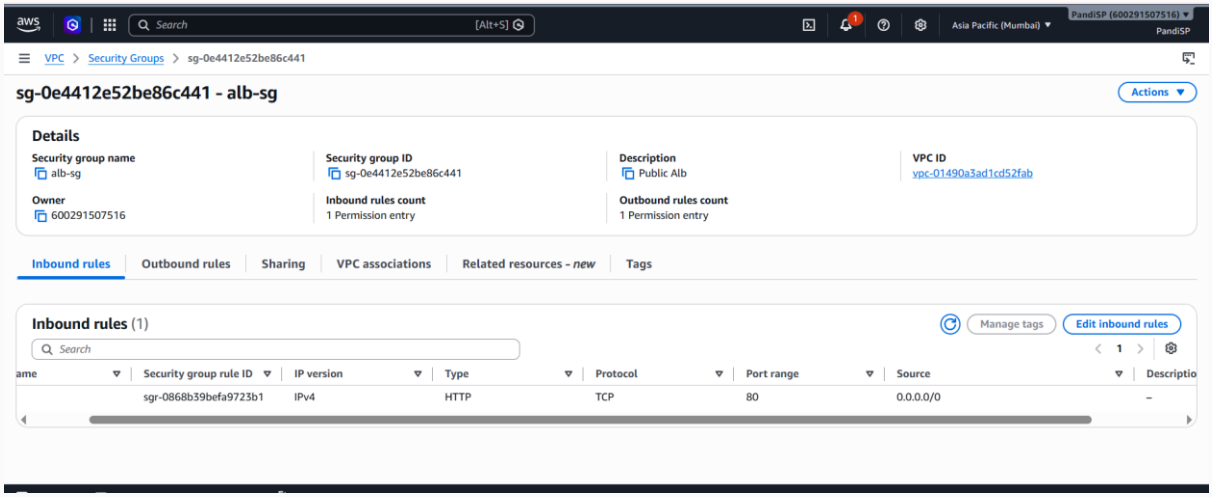
6. Task 5 - Security groups

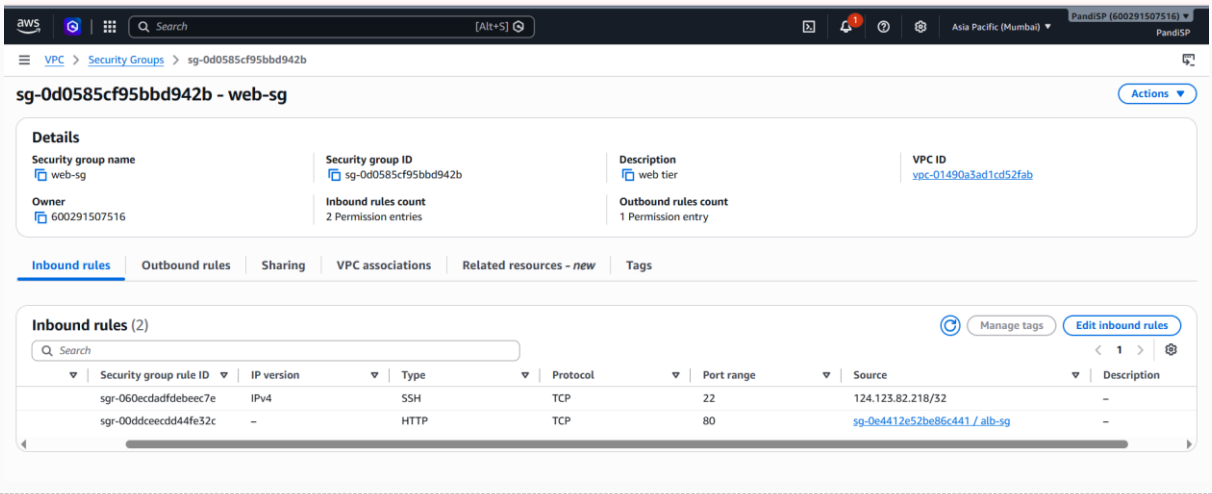
Each security group allows traffic only from the layer above it, using security group references instead of IP ranges.

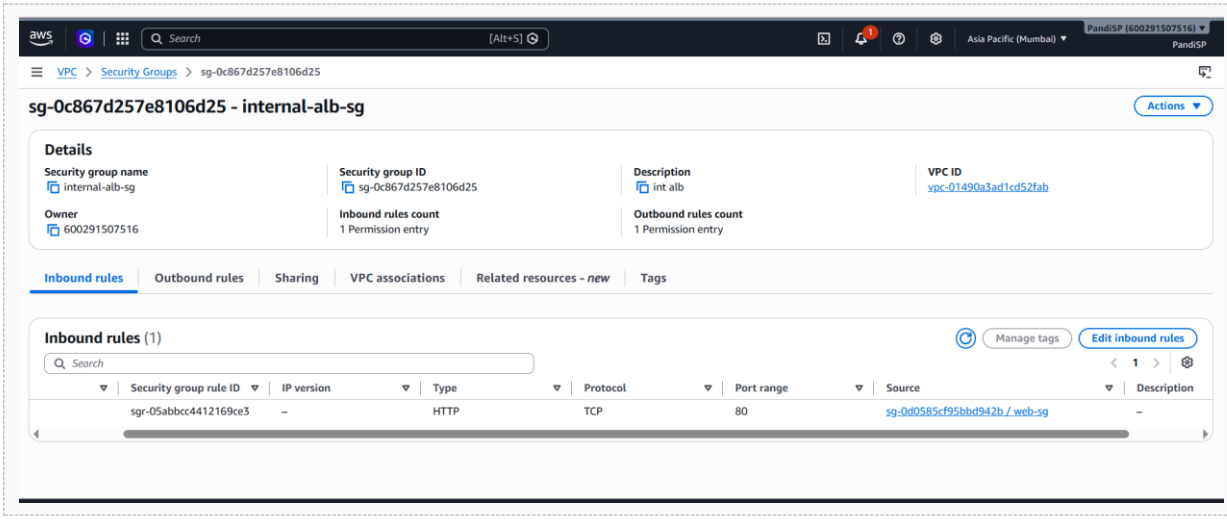
Rules configured

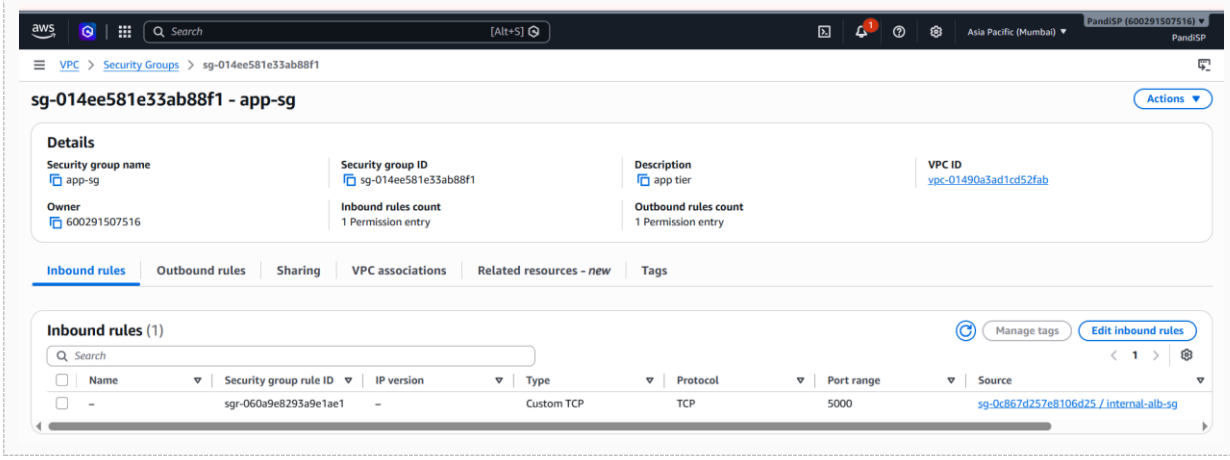
Security group	Protocol	Port	Source
alb-sg	HTTP	80	0.0.0.0/0 (internet)
web-sg	HTTP	80	alb-sg
web-sg	SSH	22	Administrator IP /32 (optional)
internal-alb-sg	HTTP	80	web-sg
app-sg	TCP	5000	internal-alb-sg
db-sg	MySQL/Aurora	3306	app-sg

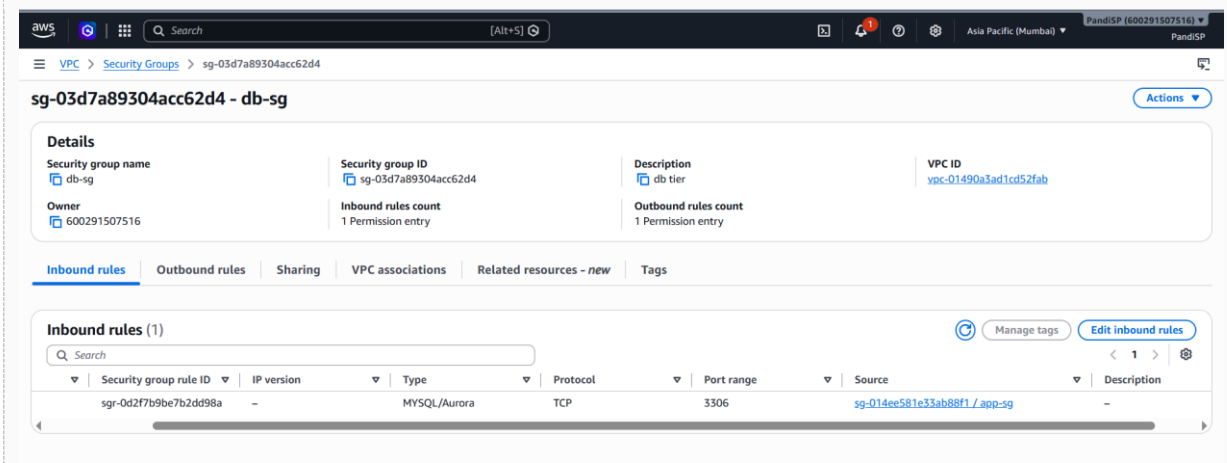
Evidence






Search
[Alt+S]
Asia Pacific (Mumbai)
PandiSP (600291507516)
PandiSP
VPC > Security Groups > sg-0c867d257e8106d25
sg-0c867d257e8106d25 - internal-alb-sg
Actions
Details
Security group name
internal-alb-sg
Security group ID
sg-0c867d257e8106d25
Description
int alb
VPC ID
vpc-01490a3ad1cd52fab
Owner
600291507516
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Search
Security group rule ID
IP version
Type
Protocol
Port range
Source
Description
sgr-05abbcc4412169ce3
-
HTTP
TCP
80
sg-0d0585cf95bbd942b / web-sg
-


Search
[Alt+S]
Asia Pacific (Mumbai)
PandiSP (600291507516)
PandiSP
VPC > Security Groups > sg-014ee581e33ab88f1
sg-014ee581e33ab88f1 - app-sg
Actions
Details
Security group name
app-sg
Security group ID
sg-014ee581e33ab88f1
Description
app tier
VPC ID
vpc-01490a3ad1cd52fab
Owner
600291507516
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Search
Name
Security group rule ID
IP version
Type
Protocol
Port range
Source
-
sgr-060a9e8293a9e1ae1
-
Custom TCP
TCP
5000
sg-0c867d257e8106d25 / internal-alb-sg


Search
[Alt+S]
Asia Pacific (Mumbai)
PandiSP (600291507516)
PandiSP
VPC > Security Groups > sg-03d7a89304acc62d4
sg-03d7a89304acc62d4 - db-sg
Actions
Details
Security group name
db-sg
Security group ID
sg-03d7a89304acc62d4
Description
db tier
VPC ID
vpc-01490a3ad1cd52fab
Owner
600291507516
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Search
Security group rule ID
IP version
Type
Protocol
Port range
Source
Description
sgr-0d2f7b9be7b2dd98a
-
MYSQL/Aurora
TCP
3306
sg-014ee581e33ab88f1 / app-sg
-

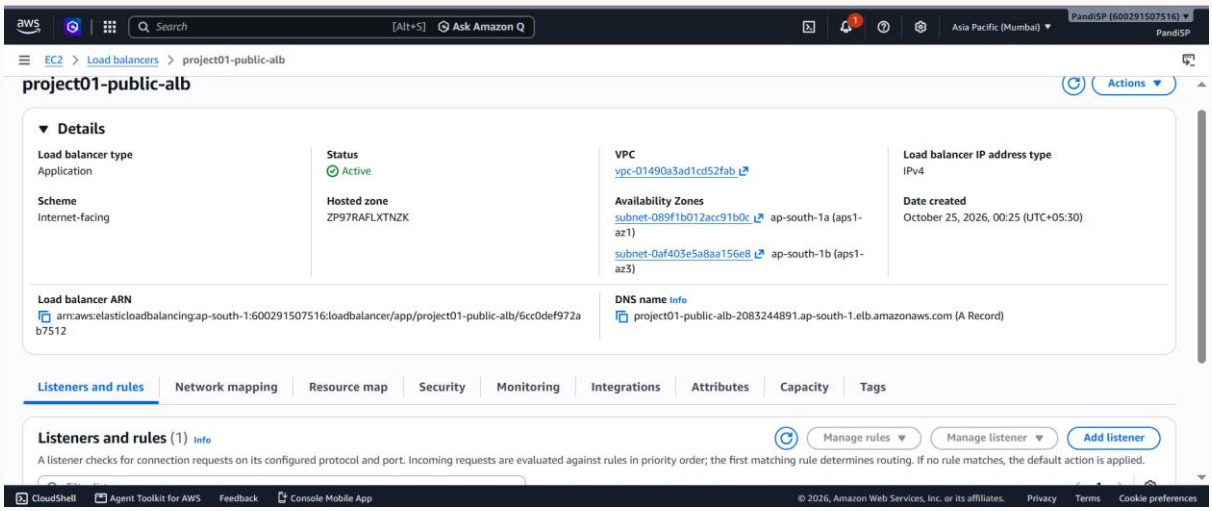
7. Task 6 - Public Application Load Balancer

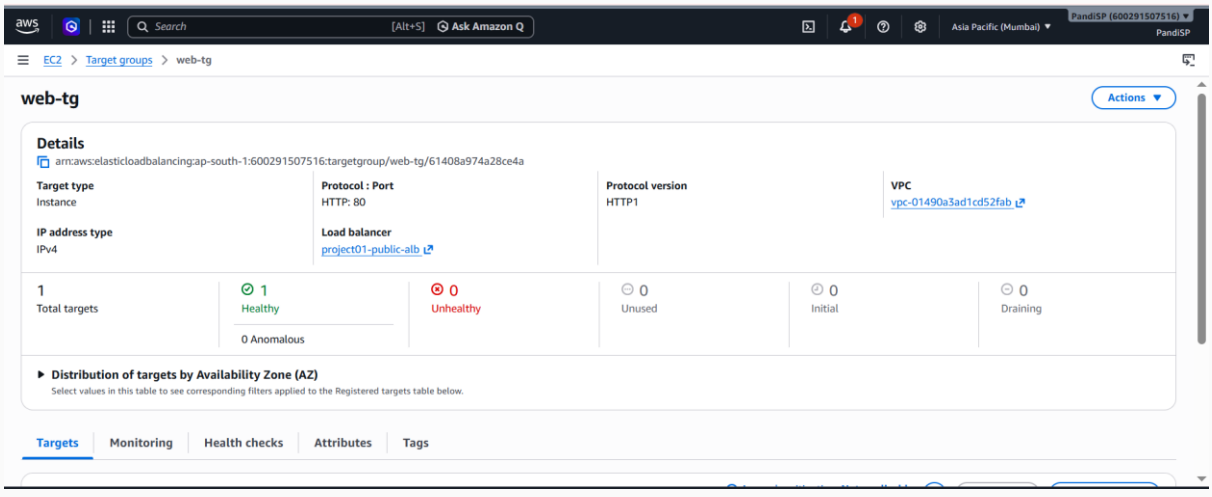
Expose the web tier to the internet through an internet-facing load balancer.

Steps performed

- Created target group web-tg (HTTP, port 80, health check path /) and registered web-server-a.
- Created internet-facing ALB project01-public-alb in web-public-a and web-public-b with security group alb-sg and an HTTP:80 listener forwarding to web-tg.
- Confirmed the target is Healthy.

Evidence





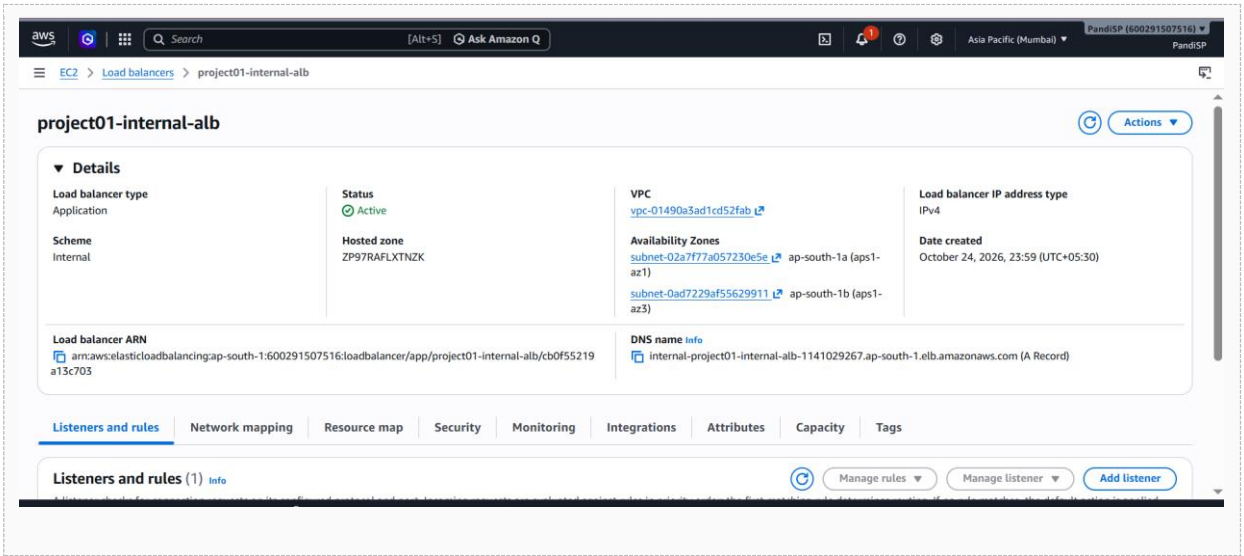
8. Task 7 - Internal Application Load Balancer

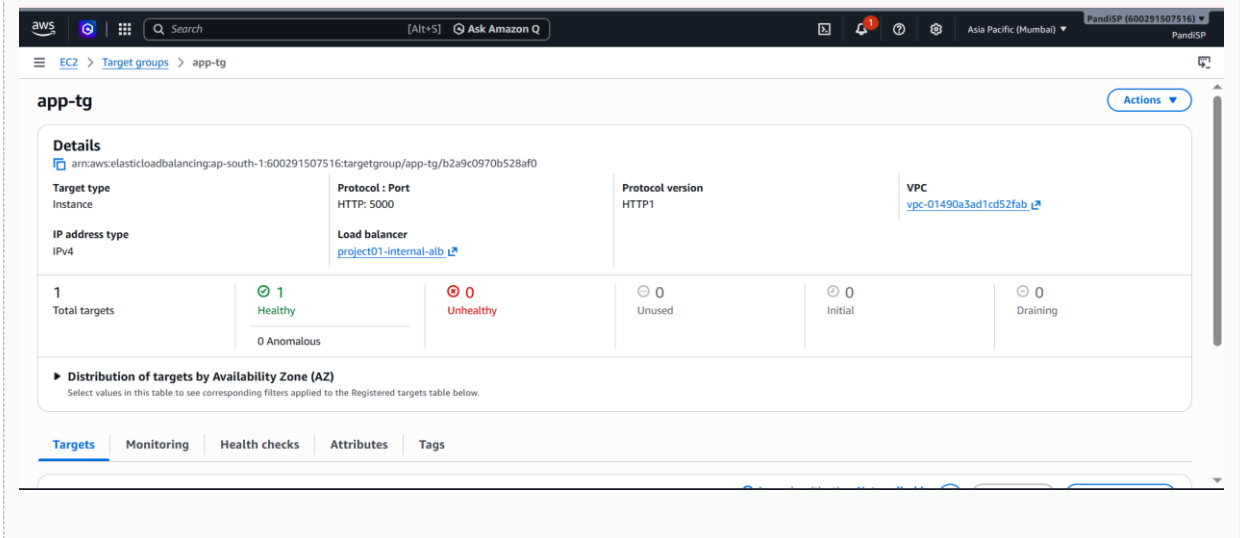
Route traffic from the web tier to the application tier privately.

Steps performed

- Created target group app-tg (HTTP, port 5000, health check path /) and registered app-server-a.
- Created an internal ALB (name cannot start with 'internal-') in app-private-a and app-private-b with security group internal-alb-sg and an HTTP:80 listener forwarding to app-tg.
- Confirmed the target is Healthy.

Evidence





9. Task 8 - Nginx reverse proxy

Serve the front end and forward API calls to the internal load balancer.

Steps performed

- Copied the project nginx.conf to /etc/nginx/nginx.conf.
- Set proxy_pass to the internal ALB DNS name (keeping the trailing slash so /api/submit reaches Flask as /submit).
- Validated the configuration and restarted Nginx.
- Tested the full chain from the web server using curl.

Key commands

```
sudo cp Three-Tier-Architecture/nginx.conf /etc/nginx/nginx.conf
grep -n "proxy_pass" /etc/nginx/nginx.conf
sudo nginx -t
sudo systemctl restart nginx
curl -X POST localhost/api/submit -H "Content-Type: application/json" -d
'{"name":"Web","email":"w@w.com","feedback":"Via Nginx and ALB"}'
# returns {"message":"Feedback submitted successfully!"}
```

Evidence



10. Task 9 - Connect Flask to RDS

Point the application at the database and run it as a service.

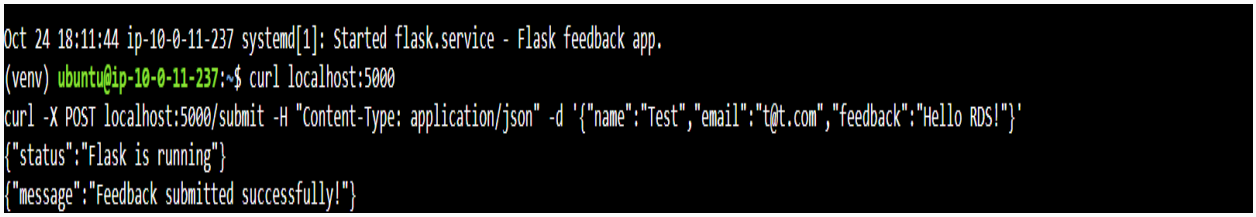
Steps performed

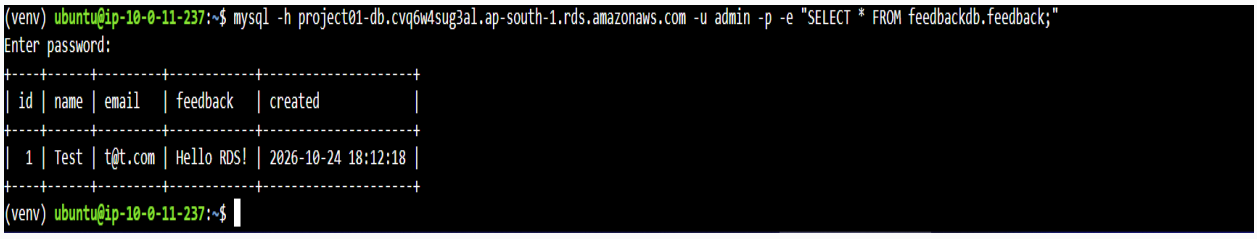
- Updated DB_CONFIG in app.py with the RDS endpoint, user, password and database name.
- Created a systemd service (flask.service) so the application starts automatically and restarts on failure.
- Posted a test record and confirmed it was stored in RDS.

Key commands

```
sudo systemctl enable --now flask
sudo systemctl status flask --no-pager
curl -X POST localhost:5000/submit -H "Content-Type: application/json" -d
'{"name":"Test","email":"t@t.com","feedback":"Hello RDS!"}'
mysql -h <RDS-ENDPOINT> -u admin -p -e "SELECT * FROM feedbackdb.feedback;"
```

Evidence





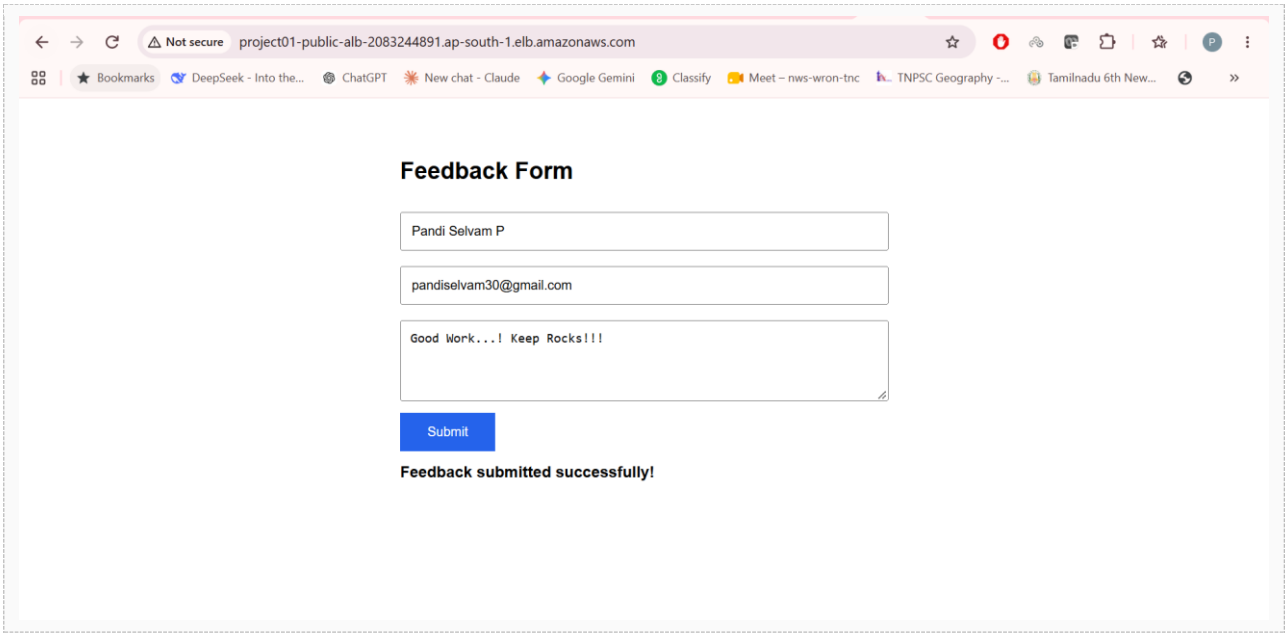
11. Task 10 - End-to-end validation

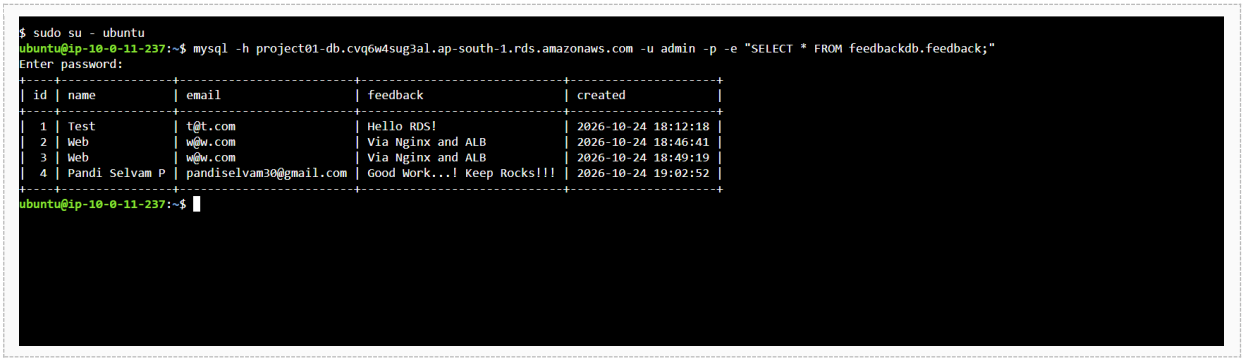
Prove that a form submission travels through all three tiers and is stored in the database.

Steps performed

- Opened the public ALB DNS name in a browser and loaded the Feedback Form.
- Submitted the form and received the success message.
- Queried the feedback table from the application server and confirmed the new record.

Evidence





12. Bonus - Auto Scaling

Auto Scaling groups were configured for the web and application tiers so capacity can adjust automatically and failed instances are replaced. This section was started and could not be completed because of an account vCPU limit, described below.

Steps performed

- Created AMIs (web-ami and app-ami) from the configured web and application instances. Flask runs as a systemd service, so it starts automatically on every new instance.
- Created launch templates (web-lt and app-lt) with the matching AMI, security group (web-sg / app-sg) and IAM instance profile.
- Created the web Auto Scaling group (web-asg) across web-public-a and web-public-b, attached to web-tg (min 1, desired 2, max 3). Instances launched from the AMI and registered with the target group.
- Created the application Auto Scaling group (app-asg) across app-private-a and app-private-b, attached to app-tg (min 1, desired 2, max 3).
- Limitation: the account allows 8 vCPUs for this instance family. The original web and app servers plus the two web-asg instances already used all 8 (each t3.micro uses 2), so app-asg could not launch. The scaling activity shows the vCPU limit error. Replacing the original servers with group instances frees capacity, but full verification was not completed.

Evidence

Status	Description	Cause	Start time	End time
Failed	Launching a new EC2 instance. Status Reason: You have requested more vCPU capacity than your current vCPU limit of 8 allows for the instance bucket that the specified instance type belongs to. Please visit http://aws.amazon.com/contact-us/ec2-request to request an adjustment to this limit. Launching EC2 instance failed.	At 2026-10-24T21:30:39Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 October 25, 03:00:40 AM +05:30	2026 October 25 03:00:40 AM +05:30
	Launching a new EC2			

Auto Scaling activity for app-asg: launch failed because of the account's 8 vCPU limit.